

Relatório Técnico Integrado — Reestruturação Arquitetural e Automação de Testes

Projeto: CSApp Backend API (csapp-back)

Escopo: Refatoração estrutural + implementação de testes automatizados + correções críticas

1. Resumo Executivo

Este relatório consolida duas frentes estratégicas aplicadas ao backend do CSApp:

1. **Refatoração arquitetural completa**
2. **Implementação de cobertura total de testes automatizados**

A aplicação evoluiu de um modelo acoplado e com débitos técnicos para uma arquitetura modular, testável, resiliente e escalável, alinhada com padrões modernos de engenharia de software.

 Resultado consolidado:

- ☒ **21 módulos** reestruturados
 - ☒ **37 test suites**
 - ☒ **203 testes** automatizados
 - ☒ **0 falhas** (100% passando)
 - ☒ Cobertura funcional completa
 - ☒ Correção de vulnerabilidades críticas
 - ☒ Eliminação de gargalos de performance (N+1)
-

2. Evolução Arquitetural

2.1 Situação Anterior

A aplicação seguia um padrão MVC acoplado, onde:

- Controllers continham regra de negócio
- Acesso ao banco era feito diretamente
- Alta duplicação de código (try/catch)
- Baixa testabilidade
- Alto risco de regressão

2.2 Nova Arquitetura: Modular em Camadas

A aplicação foi reestruturada para o padrão: **Router** → **Controller** → **Service** → **Repository**

◇ Responsabilidades:

- **Router:** Define rotas e middlewares (JWT).
- **Controller (Thin Controller):** Orquestra requisição/resposta HTTP; não contém regra de negócio.

- **Service:** Contém TODA lógica de negócio (validações, cálculos, regras e fluxos).
- **Repository:** Comunicação exclusiva com banco (Sequelize); isolamento da camada de persistência.

2.3 Estrutura Modular

```
src/modules/{modulo}/  
├─ service.js  
├─ controller.js  
├─ repository.js  
├─ routes.js  
└─ *.test.js
```

💧 Benefícios:

- Baixo acoplamento
- Alta coesão
- Testabilidade total
- Escalabilidade horizontal por módulo

3. ⚙️ Infraestrutura e Resiliência

3.1 Global Error Handler

- Centralização de erros da aplicação.
- Evita crash do servidor.
- Padroniza resposta HTTP.
- Protege contra vazamento de stacktrace.

3.2 catchAsync

- Remove repetição de try/catch.
- Encapsula controllers async.
- Redireciona erros automaticamente.

3.3 Preparação para Testes

Alterações críticas:

- Exportação do app para testes.
- Bloqueio de `app.listen()` em ambiente de teste.
- Desativação de CRONs no Jest.
- Uso de `if (process.env.NODE_ENV !== "test") { ... }`.

4. 🛠️ Automação de Testes

4.1 Stack Utilizada

- **Jest** (unitário)
- **Supertest** (integração HTTP)
- **jest.mock** (isolamento de dependências)
- **JWT mockado** para autenticação

4.2 Cobertura

- **203 testes**
- **37 suites**
- **21 módulos**

Tipos de teste:

- ☒ Unitários (Service)
- ☒ Integração (Controller/E2E)
- ☒ Regras de negócio
- ☒ Edge cases
- ☒ Segurança (401)
- ☒ Fluxos completos

4.3 Padrões de Teste

Tipo	Descrição
Success	Caminho feliz
Error	Erros esperados
Validation	Regras de negócio
Edge Case	Cenários extremos
Business	Regras específicas
Rollover	Lógica temporal
Graceful	Falhas silenciosas

4.4 Geração Automatizada

- **Script:** `scripts/generateCrudTests.js`
- **Capaz de gerar:** CRUD completo, testes de relacionamento e mocks automáticos.

5. 🐛 Correções Críticas e Vulnerabilidades

5.1 Autenticação JWT

- **Problema:** `!partes.length === 2`
- **Impacto:** Possível bypass de validação.
- **Correção:** `partes.length !== 2`.

5.2 Falhas de Persistência (Gravíssimo)

- **Problema:** Falta de `await` em operações críticas.
- **Impacto:** API retornava sucesso sem persistir dados.
- **Correção:** Sincronização total com `await`.

5.3 Inconsistências de Rotas

- Correção de endpoints divergentes.
- Ajuste de verbos HTTP.
- Padronização REST.

5.4 Tratamento de Erros

- Substituição de `Error` por `AppError`.
 - Integração com `errorHandler` global.
-

6. 🚀 Otimizações de Performance

6.1 Problema: N+1 Queries

- **Antes:** Loop com múltiplas queries; complexidade $O(n)$; alto custo de I/O.
- **Depois:** Eager Loading (include Sequelize); JOIN único.
- 📦 **Ganho:** Redução de até 95% nas queries.

6.2 Módulos Otimizados

- Vencimento de contratos
 - Relatórios
 - Vínculo vendedor ↔ cliente
-

7. 🧠 Regras de Negócio Críticas Refinadas

7.1 Cascade Inactivation

- **Fluxo:** Grupo → Clientes → Contratos.
- Totalmente controlado no Service; consistência garantida.

7.2 Classificação de Clientes

- Regra de unicidade (`tipo_categoria`).
- Validação forte no Service.

7.3 Reset de Senha

- Validação de hash UUID.
- Controle de expiração.
- Limpeza automatizada.

7.4 Histórico (Snapshot)

- Controle de execução.
- Reuso de execução pendente.
- Rollback transacional.
- Tratamento de falhas.

7.5 Relatórios

- Cálculo inteligente de vencimento.
- Tratamento de datas inválidas, duração indeterminada e rollover temporal.

8. Impacto Técnico (Antes vs Depois)

Critério	Antes	Depois
Arquitetura	Acoplada	Modular
Testes	Inexistentes	203 testes
Segurança	Vulnerável	Hardened
Performance	N+1 queries	Otimizado
Manutenibilidade	Baixa	Alta
Escalabilidade	Limitada	Alta
Confiabilidade	Baixa	Alta

9. Pronto para o Futuro

O sistema agora está preparado para:

- ☒ CI/CD (pipeline automatizado)
- ☒ Testes de regressão contínuos
- ☒ Escala modular
- ☒ Onboarding de devs facilitado
- ☒ Evolução segura de features

10. Conclusão

A transformação aplicada no backend do CSApp elevou o sistema para um nível de maturidade comparável a aplicações corporativas modernas.

A combinação de arquitetura modular, isolamento de responsabilidades, cobertura completa de testes, correções críticas e otimizações de performance resulta em um backend **robusto, confiável, seguro e preparado para evolução contínua.**